



VISIONGAIA TECHNOLOGY · SECURITY ARCHITECTURE

GeDefense Mobile

Master Architecture Datasheet + Technical Master Map

Architecture · Security · Supply Chain · Runtime · UI · Module Map

RELEASE BASELINE	0.27.8-beta.6
VERSIONCODE	55 · VC55-KeyLifecycle-FINAL
ANALYSIS	2026-09-23
ORGANIZATION	VisionGaiaTechnology (VGT)

English translated / reflowed edition

EDITORIAL NOTE

This edition translates and reflows the substantive content of the German 86-page master dossier into a more compact document. Technical identifiers, file paths, protocol names and source references remain unchanged. The German source dossier captures the VC55 architecture baseline and therefore still contains historical references such as Go 1.23.x and the corresponding source-file counts. The current repository may contain later hardening within the same VersionCode; `TOOLCHAINS.lock`, `SECURITY.md`, `CHANGELOG.md` and the source tree are authoritative for the current implementation.

Contents

Part I · Architecture Datasheet

- A. Executive Architecture Summary
- B. Product Identity and System Purpose
- C. Functional Scope
- D. Architecture Model
- E. Component Architecture
- F. Internal Dependencies
- G. Third-Party Dependencies / Supply Chain
- H. Dependency Summary
- I. Languages and Technology Stack
- J. Code Metrics
- K. Security Architecture
- L. Trust Model
- M. Threat Model
- N. Cryptography
- O. Authentication and Authorization
- P. Data Architecture
- Q. Data Flows
- R. Network Architecture
- S. OS and Platform Integration
- T. Process, Thread and Concurrency Model
- U. Error Handling and Resilience
- V. Update and Release Architecture
- W. Build and Supply-Chain Security
- X. Privacy and Telemetry
- Y. Logging, Audit and Evidence
- Z. Performance Architecture
- AA. Scalability and Capacity Limits
- AB. Configuration and Profiles
- AC. Interfaces and APIs
- AD. File Formats and Protocols
- AE. Testing and Verification

AF. Security Testing / Adversarial Coverage
AG. Quality and Maintainability
AH. Architecture Rules and Invariants
AI. Security Invariants
AJ. External Services and Infrastructure
AK. Offline Capability
AL. Attack Surface Matrix
AM. Hardening
AN. Secrets Management
AO. Privilege Model
AP. Compatibility
AQ. Packaging and Distribution
AR. Licensing
AS. Documentation Status
AT. Known Limitations
AU. Technical Debt
AV. Open Security Risks
AW. Architecture Strengths
AX. Architecture Boundaries
AY. Maturity Matrix
AZ. Architecture Factsheet

Part II · Architecture & Technical Master Map

1. Global Architecture Tree
 2. Mapping Files to Architecture
 3. Module Documentation
 4. Dashboard in Detail
 5. UI Architecture
 6. API Architecture
 7. Data Flows
 8. Shared / Core Files
 9. Architecture Relations
 10. File References
 11. Architecture Findings
-

Part I · Architecture Datasheet

A. Executive Architecture Summary

Technical facts dashboard

Attribute	Value / Status	Evidence basis
Product	GeDefense Mobile	app/src/main/res/values/strings.xml
Internal product name	VGT-GeDefense-Mobile	settings.gradle.kts
Category	On-device endpoint protection (EDR/XDR), mobile firewall and zero-trust network shield	ARCHITECTURE.md , README.md
Primary purpose	Fully autonomous, cloud-independent network, threat and device protection on Android	SECURITY.md , Manifest
Version	0.27.8-beta.6	VERSION , app/build.gradle.kts
VersionCode	55	VERSION_CODE , app/build.gradle.kts
Release channel	Public Beta / Pre-Production Security Candidate	CHANGELOG.md , release docs
Platform	Android API 29-36 (Android 10 through Android 16+)	Gradle config
ABIs	arm64-v8a and x86_64	Gradle / toolchain lock
ELF page alignment	>= 16 KiB (0x4000) PT_LOAD alignment for both ABIs	Native build gate
Languages	Kotlin, Go, Python audit tooling, Bash/PowerShell	source inventory
Runtime model	Android Native SDK + in-tree Go runtime helper	DEPENDENCIES.md
Architecture	Two-process micro-engine: Kotlin host + process-isolated Go netstack	ARCHITECTURE.md , TRANSPORT-DESIGN.md
Security model	Zero-trust local-first, least privilege, fail-closed, non-MITM	security docs
Network model	Single VpnService owner, Direct L4 or WireGuard L3 egress	transport source
At-rest storage	12 cryptographically isolated vault domains in noBackupFilesDir	vault docs
At-rest crypto	AES-256-GCM (VGTVLT01), 96-bit nonce, 128-bit tag, AAD binding, HMAC-SHA-256	core crypto
Key custody	Installation-stable AndroidKeyStore HMAC-SHA-256 hardware root vgt.gedefense.mobile.vault.root-prf.v1	PersistentVaultKeys.kt

Attribute	Value / Status	Evidence basis
Transit crypto	WireGuard Noise IKpsk2 with ChaCha20-Poly1305, Curve25519, BLAKE2s	vendored <code>wireguard-go</code>
Hybrid signatures	ECDSA P-256 plus optional ML-DSA-87 on Android 17+ / KeyMint 5	<code>HybridArtifactSigner.kt</code>
Update model	Atomic one-way vault-generation migration; encryption custody separated from APK hash	vault migration code
Android runtime dependencies	0 external libraries in <code>:app</code> and <code>:core</code>	Gradle config
Go runtime dependencies	pinned, vendored WireGuard and selected <code>golang.org/x/*</code> modules	<code>netstack/go.mod</code> , <code>third_party/go/</code>

What the system is

GeDefense Mobile is a highly integrated, entirely local Android endpoint-protection platform. It combines advanced intrusion detection and XDR, behavioral network analysis, packet filtering, offline threat detection and cryptographically protected evidence without requiring cloud analysis or external telemetry services.

Its central goal is to reduce the opacity and exposure of mobile platforms to hidden data egress, tracking, malicious application communications and command-and-control endpoints. Unlike security products that upload security telemetry or break TLS in order to inspect payloads, GeDefense uses a non-MITM design and makes policy decisions on the device.

The intended users include security-conscious individuals, researchers, regulated organizations, investigative journalists and high-risk environments. GeDefense uses Android's `VpnService` as a single network enforcement plane while preserving strict separation between the Android control plane and the packet-processing engine.

The system uses a two-process model: a Kotlin Android host owns UI, lifecycle, policy and Android Keystore access; a separate Go process, **GaiaNet V2**, processes L3/L4 traffic. The processes communicate through private Unix-domain sockets and pass file descriptors using `SCM_RIGHTS`, avoiding JNI in the packet hot path.

The network layer supports direct kernel-socket egress and an embedded `wireguard-go` L3 tunnel. In WireGuard mode there is no blanket application-UID bypass: only the exact upstream UDP transport sockets are handed to Android and explicitly bound/protected.

The Secure Telemetry Vault separates security data into independent cryptographic domains. VC55 introduced update-stable key custody rooted in a persistent AndroidKeyStore HMAC root and deliberately decoupled data encryption from APK/version identity.

B. Product Identity and System Purpose

- **Product:** GeDefense Mobile
- **Family:** VisionGaiaTechnology Security & Defense Systems
- **Organization:** VisionGaiaTechnology (VGT)
- **Category:** Mobile endpoint protection / EDR / XDR / firewall / network defense
- **Primary use:** autonomous protection of mobile traffic, blocking malicious endpoints, identifying risky applications and hardening Android security posture

- **Secondary uses:** LAN discovery, passive port-sentinel decoy detection, Device Owner administration in TITAN mode and cryptographically protected evidence
- **Platform:** Android API 29 through API 36+
- **Operating model:** standalone and local-first; core protection remains functional without a backend account
- **Privileges:** primarily a normal application using standard Android APIs; optional Device Owner elevation for managed-device policies
- **UI:** native Android View system; no WebView security plane
- **Background services:** foreground VPN service plus bounded jobs for Threat Intelligence, Integrity and Resilience
- **External infrastructure:** no mandatory VGT backend; optional HTTPS retrieval of public threat/ASN datasets

Core product promises

1. **Data sovereignty.** Security telemetry and evidence are not sent to VGT for cloud analysis.
2. **Deterministic security.** Threat lookups and block decisions are local and use bounded structures.
3. **Fail-closed integrity.** Critical security subsystem failure is not silently converted into unprotected operation.
4. **Update resilience.** Application updates must not inherently destroy encryption custody or produce false tamper results.

Explicit non-goals

- no TLS interception proxy and no automatically installed interception CA;
- no cloud antivirus file upload;
- no claim of protection against a fully compromised kernel/root environment;
- no execution of unknown malware inside an on-device VM sandbox;
- no silent universal package removal without Device Owner privileges.

C. Functional Scope

Network and policy plane

- **GaiaNet V2 L3/L4 Engine:** userspace TCP/UDP/IP handling in an isolated Go process.
- **Direct Egress:** approved flows leave through kernel sockets.
- **WireGuard L3 Egress:** full-tunnel path using vendored `wireguard-go`.
- **WireGuard Strict Mode:** cooperates with Android Always-on / Lockdown and forbids fallback to Direct when strict policy requires the secure tunnel.
- **Full Flow Protection:** captures and evaluates device traffic through TUN routes (`0.0.0.0/0` , `::/0`).
- **Selective Route Mode:** routes policy-selected threat prefixes while reducing blast radius.

Security engines

- **Threat Intelligence Engine:** local prefix matching against integrated feeds with explicit block/correlate/annotate authority.
- **InstallGuard FlowGate:** pre-egress package gate for newly installed applications.

- **InstallGuard Fast/Deep Scan:** bounded staged app analysis.
- **Telemetry Shield:** local filtering of known tracking/analytics/device-telemetry domains and endpoints using selectable profiles.
- **Encrypted-DNS Control:** non-MITM DoT/DoQ controls on port 853 in strict policy.
- **App Risk Scanner:** signer drift, permission/capability, tracker and evidence analysis.
- **Storage Malware Scanner:** bounded, read-only inspection of user-accessible areas.
- **Network Discovery:** limited local-LAN ARP/mDNS/SSDP discovery.
- **Port Sentinel:** passive decoy sockets for local scan detection; metadata only.

Data, evidence and administration

- **Secure Telemetry Vault:** 12 separated AES-GCM domains with authenticated snapshot envelopes.
- **Evidence Ledger v3:** encrypted and HMAC-chained local security evidence.
- **Hybrid Artifact Signer:** ECDSA P-256 and conditional ML-DSA-87 detached signatures.
- **Resilience Supervisor:** periodic trust-preserving reconciliation and recovery.
- **TITAN Device Owner:** optional managed-device hardening, restrictions, always-on VPN and package suspension capabilities.
- **ASN Evidence Repository:** local IP-to-ASN/operator evidence lookup.
- **Aggregate-only Diagnostics:** user-initiated export that excludes sensitive identifiers and payloads.

D. Architecture Model

GeDefense separates presentation/control responsibilities from packet processing.

```
Presentation / UI Layer
|
Application Control Plane (Kotlin host)
|
Authenticated IPC boundary (Unix-domain socket + token + SCM_RIGHTS)
|
GaiaNet V2 Data Plane (Go process)
|
Linux TUN / physical network / WireGuard transport
```

Trust boundaries

Presentation/UI renders immutable snapshots from the runtime cache and does not own cryptographic keys or packet buffers. Blocking I/O is kept off the Android main thread.

Application Control Plane owns the `GeDefenseVpnService`, serializes security-sensitive mutations on bounded executors, performs Android Keystore operations through bounded wrappers and runs integrity/recovery supervision.

IPC boundary uses app-private Unix-domain sockets. Startup is mutually authenticated by a random 32-byte token. File descriptors, rather than copied packet payloads, are transferred using `SCM_RIGHTS`.

GaiaNet Data Plane executes without Android Context/JVM bindings, reads raw IPv4/IPv6 packets from the transferred TUN descriptor and evaluates local policy before opening upstream sockets.

E. Component Architecture

Component	Responsibility	Trust	Security critical
GeDefenseVpnService	VPN lifecycle, TUN allocation, underlay binding, protected sockets	Android host	yes
NativeGaiaNet	helper process lifecycle, token handshake, descriptor transfer	Android host	yes
GaiaNet Engine	IP/TCP/UDP parsing, flow tracking, forwarding	isolated data plane	yes
WireGuardTransport	L3 encapsulation, peer state, rekeying and handshake	isolated data plane	yes
PackageEgressGate	pre-egress allow/drop decisions for quarantined packages	isolated data plane	yes
Secure Telemetry Vault	authenticated encryption/decryption of security domains	host crypto	yes
PersistentVaultKeys	stable KEK/DEK custody and domain key separation	host crypto	yes
Evidence Ledger	append-only-style authenticated security journal	host storage	yes
InstallGuard	staged inspection of newly installed packages	host scanner	yes
Resilience Supervisor	trust-preserving repair/reconciliation	host maintenance	yes
ThreatIndex	compact threat-prefix storage and lookup	read-only	yes
Telemetry Shield	unwanted tracking/telemetry DNS policy	read-only	medium
TITAN Manager	Device Owner policy application and OS hardening	Device Admin	yes

F. Internal Dependencies

```

app (Android application & UI)
|
+--> core (pure Kotlin domain/security logic)
|
`--> netstack (separate Go executable/helper)

tools/ -> build/CI verification only

```

- direct internal dependencies: `:core`, `netstack`
- no additional internal transitive submodules
- no cyclic module dependency
- `:core` is Android-independent and testable on the JVM
- `netstack` is a separate Go module: `visiongaia.dev/gedefense/mobile/netstack`

G. Third-Party Dependencies / Supply Chain

Android runtime principle

`app` depends only on the internal `:core` module. `:core` declares no external runtime library dependencies. This intentionally removes a common mobile supply-chain surface.

Vendored Go / WireGuard dependencies

Dependency	Version	Purpose	License
wireguard-go	0.0.20250522	WireGuard L3 transport and Noise protocol	MIT
golang.org/x/crypto	v0.37.0	ChaCha20/Poly1305/Curve25519 primitives	BSD-3-Clause
golang.org/x/net	v0.39.0	bounded network packet helpers	BSD-3-Clause
golang.org/x/sys	v0.32.0	Linux/Unix socket/control primitives	BSD-3-Clause
golang.org/x/term	v0.31.0	auxiliary terminal code	BSD-3-Clause
golang.org/x/text	v0.24.0	Unicode/config parsing support	BSD-3-Clause

All are stored under `third_party/go/` and wired through local `replace` directives.

Build/toolchain inputs in the source dossier

The source dossier records AGP 8.5.2, Kotlin 1.9.24, Gradle 8.7, JDK 17, Go 1.23.x and Android NDK r27c. Current repository toolchain values must be read from `TOOLCHAINS.lock`; the repository now pins Go 1.26.8.

Supply-chain assessment

- Go dependencies are physically vendored in the repository.
- builds can force `GOPROXY=off` / `GOSUMDB=off` for the vendored netstack path;
- `SOURCE-MANIFEST.sha256` binds the public source snapshot to SHA-256 hashes;
- eliminating external Android runtime libraries reduces Maven/Gradle dependency exposure.

H. Dependency Summary

- Android runtime: 0 external direct/transitive libraries in `app / core`.
- Go runtime: a small pinned vendored set used by GaiaNet/WireGuard.
- dynamically loaded third-party modules: none.
- build tools are explicitly pinned/recorded.

I. Programming Languages and Technology Stack

The dossier inventory is dominated by vendored Go source because the vendored standard/security packages are stored in-tree. First-party production code consists primarily of Kotlin for the Android control plane and Go

for the GaiaNet data plane. Python and shell/PowerShell are used for security and reproducibility tooling. Runtime C/C++ is intentionally absent from the main architecture; the `.so` extension of `libgedefense_gaianet_v2.so` is an Android extraction convention for an executable helper, not JNI loading.

J. Code Metrics

The source dossier records roughly 1.56 million physical lines including vendored source, with a much smaller first-party production surface. It highlights `GeDefenseVpnService.kt`, `AppRuntime.kt`, `AppRiskScanner.kt`, `XdrEngine.kt`, `XdrActivity.kt` and `NetworkDiscoveryScanner.kt` as the largest first-party files. The internal guideline prefers ≤ 500 LOC for stateless orchestrators, while several Android lifecycle/state owners are documented exceptions.

K. Security Architecture

1. **Confidentiality:** security telemetry/evidence is encrypted at rest using AES-256-GCM; persistent root custody is hardware-bound/non-exportable where supported.
2. **Integrity:** persisted configuration/evidence is authenticated; threat inputs are checksum/signature validated.
3. **Availability:** packet/flow/parser/scanner work is bounded to reduce DoS exposure.
4. **Authenticity:** externally sourced threat data and exported evidence use hashes/signatures where appropriate.
5. **Least privilege / compartmentalization:** Android host and Go data plane are separate; GaiaNet has no Android Keystore capability.

L. Trust Model

Untrusted inputs

- hostile/malformed network traffic;
- third-party apps and malware;
- external threat feeds;
- user-accessible shared storage;
- IPC payloads before authentication/validation.

Trusted computing base

- Android platform/Kernel within the stated threat model;
- Android KeyStore / KeyMint / StrongBox when available;
- GeDefense host control plane;
- core security algorithms and authenticated local storage.

GaiaNet is treated as a restricted component: it is necessary for packet processing but has deliberately reduced authority and no direct Keystore access.

M. Threat Model

Threats explicitly considered include malicious local apps, C2/botnet endpoints, transport attackers, malformed packets, resource exhaustion, local state tampering, update manipulation, credential/key-provider failures and hostile external files. Mitigations include pre-egress quarantine, local threat matching, WireGuard transport, bounded reassembly/flow limits, fuzzed parsers, authenticated local storage, Android signature checks and fail-closed crypto/provider behavior.

Out of scope or only partially mitigated: a fully compromised kernel/root environment, physical attacks outside Android policy boundaries, unknown DoH-on-443 hidden with ECH and rollback of an entire valid filesystem image without a trusted monotonic hardware counter.

N. Cryptography

At rest

- AES-256-GCM, 96-bit nonce, 128-bit authentication tag;
- AAD binds ciphertext to vault domain, store binding, schema and generation;
- independent HMAC-SHA-256 authentication for snapshot structures where required;
- fresh random material from platform CSPRNG.

Key hierarchy

- non-exportable installation-stable AndroidKeyStore HMAC root;
- process-local wrapping derivation;
- random per-domain root keysets wrapped at rest;
- HKDF-derived transient subkeys;
- explicit generation/version support for migration;
- sensitive in-memory byte arrays are cleared when possible.

Transit and signatures

WireGuard provides Noise IKpsk2 transport using ChaCha20-Poly1305 and Curve25519. Evidence export signatures use ECDSA P-256, with optional ML-DSA-87 where the Android hardware/API actually exposes it.

O. Authentication and Authorization

- Android user approval gates VPN establishment through `VpnService.prepare()`.
- product-level VPN disclosure is separately authenticated in local storage.
- helper startup uses a fresh mutual 32-byte token on a private Unix socket.
- privileged TITAN actions require actual Android Device Owner/admin state, not merely a local preference.
- managed trust-anchor installation requires bounded certificate validation and explicit confirmation.

P. Data Architecture

Sensitive durable state is placed under `noBackupFilesDir` and separated into independent domains. Examples include XDR events, evidence, scanner caches, behavior baselines, firewall policy, WireGuard profile state, package baselines, network-discovery history, Port Sentinel state and TITAN policy. Android backup/data-transfer extraction is disabled for these security stores.

Q. Data Flows

Full Flow

```
Android app traffic
-> /dev/tun
-> GaiaNet packet parser/state engine
-> threat/privacy/package policy
-> allow/drop/quarantine decision
-> Direct socket or WireGuard transport
-> physical network
```

Evidence

```
security event
-> XDR correlation
-> encrypted local event/evidence store
-> HMAC chain / authenticated snapshot
-> optional user-initiated signed export
```

Threat updates

```
HTTPS feed download
-> bounded validation/parsing
-> compile canonical threat index
-> atomic authenticated cache publication
-> immutable policy handed to GaiaNet
```

R. Network Architecture

GeDefense owns one Android VPN instance. Direct mode uses protected upstream kernel sockets; WireGuard mode uses an embedded userspace L3 transport. Exact upstream WireGuard socket descriptors are passed to Android for `VpnService.protect()` and physical-network binding. The design explicitly rejects whole-UID tunnel bypass.

S. OS and Platform Integration

Important Android interfaces include `VpnService`, `ConnectivityManager`, `PackageManager`, `DevicePolicyManager`, `JobScheduler`, `Storage Access Framework` and `Android KeyStore/KeyMint`. System callbacks are treated as boundaries; blocking work is handed to bounded background executors.

T. Process, Thread and Concurrency Model

- main/UI thread: rendering and lightweight lifecycle coordination only;
- `gedefense-control`: serialized transport/security state mutations;
- bounded worker pools: scans, downloads and expensive platform calls;
- Go runtime: packet/flow work with explicit limits and tested race behavior;
- no unbounded work queues are considered acceptable in security paths.

U. Error Handling and Resilience

Failures are classified by impact. Security-critical crypto/integrity/transport failures remain fail-closed. Last-known-good threat data may remain active when an update fails validation. Best-effort UI/telemetry degradation must not silently weaken enforcement.

The `ResilienceSupervisor` periodically checks vault/state integrity, package-baseline drift and helper health. Recovery is designed to preserve trust: it must not erase evidence, lift quarantine or convert a failed integrity result into success merely to restore a green UI state.

V. Update and Release Architecture

VC55 decoupled encryption custody from APK identity. The installation-stable root is created once per installation and does not intentionally include APK hash, source-manifest hash or version number in its derivation path. Integrity monitoring remains a separate concern.

Schema/key generations are migrated one way: read and authenticate the historical generation, migrate in memory, durably publish the new generation and only then retire old material.

W. Build and Supply-Chain Security

- source files are bound by `SOURCE-MANIFEST.sha256`;
- vendored Go modules support offline native builds;
- exact toolchain versions are recorded;
- release Kotlin code is minified/obfuscated and unused resources are shrunk;
- native helpers are checked for 16 KiB load-segment alignment and reproducibility.

X. Privacy and Telemetry

The dossier states a zero-product-telemetry design: no Firebase Crashlytics, Sentry, Bugsnag, advertising SDK or analytics SDK. DNS/IP observations are used locally for protection and evidence. The user-initiated diagnostic export enforces an aggregate-only profile that removes package identities, IPs, domains, file paths, payloads and other sensitive details.

Telemetry Shield is separate from GeDefense's own privacy posture: it can apply local policy to known third-party tracking/telemetry endpoints without decrypting TLS.

Y. Logging, Audit and Evidence

Evidence Ledger v3 maintains authenticated continuity through chained HMACs and encrypted record payloads. User-requested exports can be accompanied by detached signatures. The goal is tamper evidence and trustworthy local chronology, not a claim that the app can resist a fully compromised kernel controlling the process.

Z. Performance Architecture

The design uses fixed/bounded buffers in packet paths, compact prefix lookup structures and background execution for blocking control-plane work. UI animation quality may adapt to battery/memory conditions, but policy evaluation and critical evidence must not be intentionally throttled for cosmetic power savings.

AA. Scalability and Capacity Limits

The dossier records explicit limits such as:

- unacknowledged TCP data per flow: 1 MiB;
- global downstream buffering: 32 MiB;
- upstream dial workers: 64;
- active UDP flows: 1,024;
- normal TCP idle timeout: 10 minutes;
- constrained-power TCP timeout: 90 seconds;
- UDP idle timeout: 90 seconds (30 seconds in constrained power mode);
- InstallGuard Fast Verdict budget: 1.75 seconds;
- Deep Scan budget: 25 seconds;
- Network Discovery sweep: max 254 hosts / local /24 ;
- WireGuard import: 16 KiB / 128 lines.

Exceeding a limit produces an explicit bounded consequence: dropping/recycling state, keeping quarantine active, rejecting malformed input or ending a stale flow.

AB. Configuration and Profiles

Telemetry Shield profiles

1. **OFF**: no GeDefense DNS telemetry filter.
2. **CONSERVATIVE**: known tracking/advertising domains; observes encrypted-DNS indicators.
3. **BALANCED**: broader tracking/telemetry filtering; default profile in the dossier.
4. **STRICT**: strongest policy and active DoT/DoQ port-853 blocking where configured.

Secure defaults

- `allowBackup=false`
- `usesCleartextTraffic=false`
- network security configuration enforces HTTPS expectations for internal update tasks
- new/unknown apps are gated/quarantined by InstallGuard until a bounded verdict is available.

AC. Interfaces and APIs

- Helper Protocol v5 over Unix-domain socket with versioned framing and MTU negotiation.
- startup descriptor set contains the TUN plus telemetry/policy/gate/WireGuard descriptors depending on mode.
- WireGuard UAPI is internal and does not expose private keys on a CLI.
- Android APIs include `VpnService`, `ConnectivityManager`, `DevicePolicyManager` and `JobScheduler`.
- no public HTTP/REST/RPC server is exposed by GeDefense.

AD. File Formats and Protocols

- `VGTVLT01` : binary AEAD vault envelope.
- `VGTPVK01` : wrapped persistent vault-keyset format.
- `GDTI-v2` : compact threat-prefix policy/index format.
- JSON: detached signatures and aggregate diagnostics.
- local PDDL/IPtoASN binary indexes for ASN evidence.
- standard WireGuard Noise-based protocol over UDP.

AE. Testing and Verification

The dossier lists dedicated harnesses for core logic, vault/key wrapping, Privacy Shield, WireGuard, InstallGuard, resilience, startup/main-thread behavior, internationalization, source-manifest integrity and Go packet/flow behavior. Release criteria require PASS results rather than treating compilation alone as evidence.

AF. Security Testing / Adversarial Coverage

Adversarial coverage includes ciphertext/AAD tampering, parser fuzzing, Go race detection, malformed packet handling and verification that WireGuard cannot use a blanket UID escape path. Negative paths are expected to preserve security invariants.

AG. Quality and Maintainability

The architecture deliberately separates Android host, pure Kotlin core and Go data plane. Historical crypto generations are retained only as explicit migration paths. Shared algorithms live in :core ; production placeholders/TODO-style dummy paths are not considered acceptable release state.

AH. Architecture Rules and Invariants

1. no JNI in the packet hot path;
2. one Android VPN owner;
3. no network/file/crypto blocking on the main looper;
4. loaded threat policy is immutable until atomically replaced.

AI. Security Invariants

1. no secrets in normal logs, preferences or crash output;
2. clear sensitive mutable buffers when practical;
3. fail closed when key custody fails;
4. no TLS decryption/forged interception certificates in the normal protection plane;
5. no whole-UID WireGuard bypass.

AJ. External Services and Infrastructure

There is no mandatory GeDefense backend. Optional public security data sources provide threat intelligence updates. If an external feed is unavailable, protection continues using the last locally verified state instead of failing open.

AK. Offline Capability

Threat indexes, ASN evidence, app baselines and evidence are local. Packet filtering, firewall policy, Telemetry Shield, quarantine and app-risk analysis can operate without a continuous Internet connection, subject to whatever external network access the protected apps themselves require.

AL. Attack Surface Matrix

Vector	Exposure	Primary mitigation
TUN ingress	malformed IPv4/IPv6	memory-safe Go, fragment limits, fuzzed parsing
local IPC	socket manipulation	private app directory, random token, strict framing
Android intents	unauthorized activity invocation	critical components <code>exported=false</code> / protected
backup/restore	extraction of vault data	backup disabled + Android extraction rules
shared storage	hostile scan targets	read-only scanner, bounded parsing, no execution
update injection	malicious replacement	Android package signature verification + signer drift evidence

AM. Hardening

Native ELF hardening includes PIE/NX/RELRO expectations and ≥ 16 KiB page alignment. Kotlin release builds use R8/minification. Cleartext application traffic for GeDefense's own network operations is disabled. Evidence exports can carry detached signatures.

AN. Secrets Management

The master root PRF is an AndroidKeyStore HMAC-SHA-256 key. Domain roots use fresh random 256-bit material, are wrapped by the derived KEK and stored under `noBackupFilesDir/secure-vault-keysets/`. Operational subkeys are derived transiently. Keystore keys are intended to be non-exportable; hardware backing/StrongBox is used where the platform supports the required primitive reliably.

AO. Privilege Model

Normal mode runs as an unprivileged Android application UID. The Go helper receives only explicit descriptors/capabilities from the host. Optional TITAN Device Owner mode adds OS-managed capabilities such as enforced Always-on VPN, USB/debugging restrictions and package suspension, within Android's DPC boundaries.

AP. Compatibility

- minimum Android: API 29 / Android 10;
- target/compile: API 36;
- arm64-v8a primary mobile ABI;
- x86_64 for emulator/x86 targets;
- special attention is paid to aggressive OEM background behavior such as HyperOS and OneUI.

AQ. Packaging and Distribution

Release output includes a minified/signed APK and an Android App Bundle. No dynamic code loading from the Internet is part of the architecture. Native helpers are built from source and verified against packaged bytes.

AR. Licensing

The German source dossier predates the current public-repository licensing decision and describes a proprietary VGT license. The repository delivered with this translated edition contains an **AGPL-3.0-or-later LICENSE**; that repository license is authoritative for this public source tree. Vended upstream components retain their own MIT/BSD/public-domain licenses as documented in `DEPENDENCIES.md` / `SUPPLY-CHAIN.md`.

AS. Documentation Status

Architecture, security, threat model, vault design, transport, build, beta testing and release gates are documented in repository Markdown plus machine-verifiable audit scripts. This master dossier is a snapshot of the VC55 architecture baseline, not a replacement for current source-level evidence.

AT. Known Limitations

1. `PACKAGE_ADDED` timing is controlled by Android; without Device Owner, absolute pre-first-packet notification cannot be mathematically guaranteed. FlowGate narrows this gap.
2. Non-MITM design means GeDefense cannot inspect encrypted application payloads hidden inside legitimate HTTPS/DoH traffic.
3. Android does not expose a universal trusted monotonic counter for detecting replay of an entire previously valid filesystem image.

AU. Technical Debt

The dossier identifies several large Android lifecycle owners, the need for carefully prepared offline AGP caches, and local ASN asset size as maintenance considerations. These are architectural/operational debt rather than hidden runtime dependencies.

AV. Open Security Risks

- kernel/root compromise can observe or control a legitimately running process;
- unknown DoH over HTTPS/ECH may be indistinguishable from normal encrypted traffic;
- OEM Keystore provider defects can temporarily make key operations unavailable, requiring bounded fail-closed recovery behavior.

AW. Architecture Strengths

- zero third-party Android runtime-library surface;

- process isolation between Android control plane and Go data plane;
- no JNI packet hot path;
- update-stable key custody;
- early hybrid/PQ signature readiness;
- fail-closed network/security-state design;
- extensive machine-verifiable release gates.

AX. Architecture Boundaries

GeDefense depends on Android/KeyStore platform integrity. It cannot provide mathematical protection against a fully compromised unlocked device controlled at kernel level. It intentionally does not perform DPI of encrypted TLS payloads.

AY. Maturity Matrix

The dossier rates architecture, crypto, storage and the core security design as release-hardened while noting that networking/WireGuard and OEM compatibility require real-device matrices. Supply-chain and build reproducibility are treated as first-class release properties rather than paperwork.

AZ. Architecture Factsheet

Product: GeDefense Mobile

Version: 0.27.8-beta.6 / VC55

Platform: Android 10+ / API 29+

Architecture: Kotlin host + isolated Go GaiaNet data plane

Runtime Android third-party libraries: 0

At-rest crypto: AES-256-GCM + HMAC-SHA-256

Persistent key root: AndroidKeyStore HMAC root PRF

Transport: Direct L4 or embedded WireGuard L3

Privacy: local-first, non-MITM, aggregate-only diagnostics

Admin: optional Device Owner / TITAN

Build: reproducible native helper, pinned toolchain, signed artifacts, source manifest and SBOM

Part II · Architecture & Technical Master Map

1. Global Architecture Tree

```
GeDefense Mobile
├─ Presentation / UI
│   ├── MainActivity + five primary hubs
│   ├── Dashboard / Threat / Protection / Analysis / System
│   └─ module activities (Firewall, WireGuard, Privacy, Scanner, XDR, TITAN...)
├─ Application Control Plane (Kotlin)
│   ├── AppRuntime / RuntimeState / UiSnapshot
│   ├── GeDefenseVpnService
│   ├── NativeGaiaNet IPC/process bridge
│   ├── Secure Telemetry Vault / PersistentVaultKeys
│   ├── XDR / Evidence / InstallGuard / Resilience
│   └─ Android platform adapters
├─ Pure Kotlin Core
│   ├── AEAD envelope and authenticated snapshots
│   ├── threat/route/privacy indexes
│   ├── evidence ledger and network parsers
│   └─ bounded recovery/policy primitives
└─ GaiaNet V2 (Go)
    ├── IP/TCP/UDP engine
    ├── threat/privacy/package policy
    ├── helper IPC + descriptor handling
    ├── Direct transport
    └─ embedded WireGuard L3 transport
```

2. Mapping Files to Architecture

Presentation and visual components

Key visual/view files include `CyberBackgroundView.kt`, `ShieldPulseView.kt`, `VgtActionTile.kt`, `VgtIconView.kt`, `VgtProgressView.kt`, `TitanCoreView.kt`, `ScannerRadarView.kt` and `TrafficWorldMapView.kt`.

Primary screens/activities include:

- `DashboardScreen.kt`
- `ThreatScreen.kt`
- `ProtectionHubScreen.kt`
- `AnalysisHubScreen.kt`
- `SystemHubScreen.kt`
- `FirewallActivity.kt`
- `WireGuardActivity.kt`
- `PrivacyActivity.kt`
- `ScannerActivity.kt`
- `XdrActivity.kt`

- `HardeningActivity.kt`
- `BehaviorActivity.kt`
- `NetworkDiscoveryActivity.kt`
- `PortSentinelActivity.kt`
- `TitanActivity.kt`
- `EvidenceScreen.kt`
- `DiagnosticsActivity.kt`
- `SupportVgtActivity.kt`
- `SetupWizardActivity.kt`
- `VpnDisclosureActivity.kt`

Application control plane

Lifecycle/state: `GeDefenseApplication.kt`, `AppRuntime.kt`, `RuntimeState.kt`, `UiSnapshot.kt`, bounded executor/scheduler/call wrappers and `ProtectionMetrics.kt`.

Network/TUN control: `GeDefenseVpnService.kt`, `NativeGaiaNet.kt`, `ConnectionOwner.kt`, `PackageEgressGate.kt`.

Persistent stores/repositories include firewall policy, WireGuard profiles, XDR events, behavior/package baselines, malware-analysis state, approvals, network discovery, Port Sentinel, TITAN policy, scanner caches, VPN disclosure, runtime evidence, threat intelligence, ASN evidence, Geo-country, Privacy Intelligence and traffic-usage stores.

Scanning/protection engines include InstallGuard, AppRiskScanner, StorageMalwareScanner, ThreatTokenScanner, DeviceHardeningScanner, BehaviorEngine, IntegrityGuardian, PortSentinel, NetworkDiscoveryScanner, TitanPolicyManager, FeedDownloader and WireGuardConfigParser.

System/background components include ThreatIntelJobService, IntegrityJobService, ResilienceJobService, BootReceiver, PackageChangeReceiver and TITAN receivers.

Go data plane

Engine core: `main.go`, `engine.go`, `packet.go`, `build_packet.go`, `checksum.go`, `reassembly.go`, `limits.go`, `power_state.go`, `tun_writer.go`, `telemetry.go`.

Flow managers: `tcp_manager.go`, `tcp_flow_io.go`, `udp_manager.go`, `dns.go`, flow keys/IDs/limiters.

Policy: `policy.go`, `privacy_policy.go`, `package_egress_gate.go`.

IPC/process: `helper_main_process.go`, stubs/bridges, socket-binding files.

WireGuard: `wireguard_transport.go`, `wireguard_tun.go`, `wireguard_protected_bind.go`, `wireguard_flow_tracker.go`.

Cryptographic/core files

Host adapters include `SecureTelemetryVault.kt`, `PersistentVaultKeys.kt`, historical migration derivators, `AndroidSecrets.kt`, `AndroidKeystoreGate.kt`, `AndroidEvidence.kt` and `HybridArtifactSigner.kt`.

Pure JVM primitives include `AeadVaultEnvelope.kt`, `AuthenticatedSnapshotStore.kt`, `BoundedSecretKeyCrypto.kt` and `EvidenceLedger.kt`.

Core policy/data structures include `ThreatIndex.kt`, `ThreatFeed.kt`, `ThreatIntelParser.kt`, `ThreatCacheStore.kt`, `IpPrefix.kt`, `RouteCompactor.kt`, `PrivacyIntelligence.kt`, `AsnLiteIndex.kt`, `MdnsDnsCodec.kt`, `NetworkRangePlanner.kt`, `LanBehaviorEvaluator.kt`, `PortSentinelClassifier.kt`, `ManagedCaCertificateValidator.kt`, `IntegrityBaseline.kt`, `FlowTable.kt`, `TelemetryLearningPolicy.kt` and `RecoveryAttemptBudget.kt`.

Verification tooling

The dossier maps a dedicated `tools/` suite for security audit, vault, privacy, WireGuard, InstallGuard, resilience, availability, startup/ANR, main-thread I/O, runtime bootstrap, JVM class-init, source manifest, artifact audit, egress, ASN, onboarding, i18n, lint, core tests, WireGuard source checks and reproducible native build/package workflows.

3. Module Documentation

Host & Application Runtime

Purpose: initialize the Android process, own long-lived subsystems, maintain immutable runtime/UI snapshots and execute mutations asynchronously.

Capabilities: asynchronous bootstrap away from `Application.onCreate()`, dependency-root/state ownership, serialized VPN/crypto mutation, platform lifecycle handling.

Dashboard integration: provides the five primary hubs, feeds `UiSnapshot`, and orchestrates protection start/stop without disk/network work in rendering callbacks.

Network & VPN Core / Transport Plane

Purpose: sole owner of Android `VpnService`, TUN allocation, route configuration, network handover and the GaiaNet child process.

Capabilities: Full Flow/Selective routes, TUN liveness anchor, underlay binding and per-socket `VpnService.protect()` for transferred upstream sockets.

GaiaNet V2 Netstack

Purpose: memory-safe high-performance packet engine running out of process.

Capabilities: IPv4/IPv6 parsing, TCP/UDP state, threat/privacy/package decisions, bounded reassembly/flows and Direct/WireGuard transport.

Threat Intelligence & Policy Engine

Purpose: download/validate/compile public security feeds into a compact local policy with explicit authority semantics.

Data path: bounded HTTPS download → parser/normalizer → compact index → authenticated atomic cache → immutable helper policy.

Secure Telemetry Vault & Key Custody

Purpose: protect persistent security state at rest and across application updates.

Capabilities: per-domain random keysets, installation-stable root PRF, AES-GCM envelopes, HMAC snapshots, explicit generation migration, fail-closed provider handling and backup exclusion.

XDR / EDR Correlation & Incident Plane

Purpose: merge independent package, network, behavior, hardening and integrity signals into explainable local incidents rather than treating one heuristic as authority.

Capabilities: bounded event correlation/scoring, incident persistence, network quarantine, evidence timeline and optional Device Owner escalation.

InstallGuard & Scanner Plane

Purpose: reduce first-egress exposure of newly installed applications and provide bounded malware/risk scanning.

Capabilities: Fast Verdict, Deep Scan, package signer/capability analysis, local caches as non-authoritative acceleration, network quarantine before destructive actions.

Telemetry Shield & Privacy Engine

Purpose: apply local tracking/telemetry/DNS policy without breaking TLS.

Capabilities: profile-based domain policy, encrypted-DNS port controls, signed/local Privacy Intelligence and reporting into the local evidence/XDR plane.

Network Discovery & Port Sentinel

Purpose: provide bounded local-LAN visibility and passive scan evidence.

Network Discovery restricts active scanning to the attached private LAN and a bounded /24 ; Port Sentinel uses bounded decoy listeners and retains metadata rather than payloads.

TITAN Device Owner Plane

Purpose: optionally apply Android-managed hardening policies unavailable to a normal app.

Capabilities: Always-on/lockdown policy, restrictions, package suspension, Kiosk-style controls, managed CA installation with explicit validation/confirmation and optional destructive policies only after explicit administrator confirmation.

Resilience & Self-Healing Supervisor

Purpose: detect drift/corruption and restore only states that can be restored without weakening trust.

The supervisor must never erase evidence, silently accept failed integrity, or lift quarantine simply to improve health status.

Diagnostics & Support Plane

Purpose: user-controlled troubleshooting without central telemetry.

The `aggregate-only-v1` export removes package identities, domains, IPs, paths, payloads and detailed incident contents, and uses Android SAF for user-selected storage.

Setup & Onboarding Assistant

Purpose: explain the local VPN/privacy model, obtain explicit consent, guide required platform settings and initialize the protection baseline.

The original dossier describes a 10-step wizard. The current repository additionally promotes Telemetry Shield as a first-class privacy capability and keeps initial package inventory deferred until setup completion.

4. Dashboard in Detail

Cockpit / Dashboard

The dashboard reads an immutable `UiSnapshot`, shows protection health, Threat Intelligence status, blocked-event/session metrics and malware-analysis state. Activating protection goes through product disclosure, Android VPN consent, `GeDefenseVpnService`, authenticated helper startup and only then renders the protected state.

Threat Intelligence screen

Displays feed status, timestamps and geographic/intelligence context. A manual sync triggers bounded HTTPS download, parsing, authenticated cache persistence and an immutable GaiaNet policy reload.

Protection Hub

Navigation hub for firewall, WireGuard, Privacy/Telemetry Shield, scanner, Port Sentinel and Network Discovery modules.

Analysis Hub

Forensic/XDR hub showing unresolved incidents, scores and routes to XDR, behavior, hardening and evidence views.

System Hub

Provides TITAN, diagnostics, setup and maintenance views, including actual Device Owner state rather than trusting a preference bit.

Firewall Manager

Per-app network policy is stored in an encrypted vault domain and atomically synchronized with active enforcement.

WireGuard Manager

Parses bounded WireGuard configuration, persists protected profile state and performs the exact-socket protection handshake before marking the L3 tunnel active.

Privacy & Telemetry Shield

Offers profile selection and local tracker/telemetry visibility. STRICT can enforce configured encrypted-DNS controls. Changes propagate through the host into GaiaNet privacy policy.

Scanner & Quarantine

Runs package/storage analysis on bounded worker pools, keeps threat decisions current rather than trusting stale caches, and uses network quarantine as the immediate safe response.

XDR Incident Explorer

Decrypts local incident state, renders correlation explanations/timelines and can request immediate network isolation. Device Owner-only actions remain separately authorized.

Hardening / Behavior / Evidence / TITAN / Diagnostics

These views expose live OS posture, behavioral drift, cryptographically protected chronology, managed-device policy state and sanitized diagnostics respectively.

5. UI Architecture

The UI is a native Android View hierarchy with a dark glass/cyber visual system, reusable VGT controls and adaptive rendering. Presentation reads cached immutable snapshots instead of performing direct storage/network operations. Security status is communicated with text/iconography rather than color alone, and critical actions require explicit confirmation when destructive or privilege-sensitive.

6. API Architecture

GeDefense exposes no public HTTP API. Internal APIs are Kotlin interfaces/services, Android platform APIs and the versioned Unix-socket helper protocol. WireGuard configuration uses an internal UAPI stream. Inter-process boundaries validate versions, sizes, descriptor counts and authentication tokens before accepting state.

7. Data Flows

Package installation

```
PACKAGE_ADDED
-> InstallGuard Fast Verdict
-> quarantine / PackageEgressGate
-> optional Deep Scan
-> XDR + Evidence
-> release or continued block according to verified outcome
```

Local telemetry policy

```
DNS/flow metadata in GaiaNet
-> Privacy Intelligence profile
-> allow/block decision
-> bounded security event
-> optional XDR/Evidence correlation
```

Device integrity

```
scheduled/live platform observations
-> IntegrityGuardian / baseline verification
-> authenticated local state
-> XDR/Evidence on degradation
```

8. Shared / Core Files

The dossier identifies the pure Kotlin core as the shared deterministic layer for encryption envelopes, authenticated snapshots, evidence chains, threat indexes, IP prefixes, route compaction, privacy policy primitives, ASN indexing, bounded mDNS/DNS parsing, LAN planning/behavior evaluation, Port Sentinel classification and recovery budgets.

`AppRuntime.kt`, `GeDefenseVpnService.kt` and `NativeGaiaNet.kt` are central host-side ownership points. Their size is justified in the dossier by lifecycle/state ownership and the desire to avoid splitting a single mutation state machine across competing threads.

9. Architecture Relations

Whole-system relationship

```
Android UI
-> AppRuntime / GeDefenseVpnService / Resilience
-> Secure Vault + Evidence + InstallGuard
-> authenticated Unix IPC
-> GaiaNet packet/policy engine
-> Direct sockets OR embedded WireGuard
-> physical network
```

Android KeyStore/KeyMint remains on the host side. GaiaNet receives explicit descriptors and policy material, not general Android privileges.

Navigation relationship

`MainActivity` hosts five primary tabs: Dashboard, Threat, Protection, Analysis and System. Sub-activities provide firewall, WireGuard, Privacy, scanning, network discovery, Port Sentinel, XDR, hardening, behavior, evidence, TITAN, diagnostics, support and setup.

Backend/evidence relationship

Packet/app/system events feed dedicated engines; XDR correlates evidence; local encrypted stores persist security state; threat cache replacement remains a separate atomic policy-update path.

10. File References

Important implementation anchors include:

- async bootstrap: `AppRuntime.kt`, `StartupActivity.kt`;

- TUN allocation/liveness: `GeDefenseVpnService.kt`, `NativeGaiaNet.kt`;
- SCM_RIGHTS protocol: `NativeGaiaNet.kt`, `netstack/helper_main_process.go`;
- WireGuard socket protection: `wireguard_protected_bind.go`, `NativeGaiaNet.kt`, `GeDefenseVpnService.kt`;
- update-stable key custody: `PersistentVaultKeys.kt`, `AndroidSecrets.kt`;
- pure-JVM AEAD/AAD: `AeadVaultEnvelope.kt`;
- InstallGuard/FlowGate: `InstallGuard.kt`, `package_egress_gate.go`;
- DoT/DoQ policy: `privacy_policy.go`, `PrivacyIntelligence.kt`;
- hybrid signatures: `HybridArtifactSigner.kt`;
- single-flight recovery: `ResilienceSupervisor.kt`, `RecoveryAttemptBudget.kt`.

11. Architecture Findings

The source dossier identified several historical artifacts:

1. `MoreScreen.kt` and `SecurityScreen.kt` were old dashboard implementations with no active callers and were recommended for later cleanup.
2. `settings.gradle.kts.orig` was a stale backup file and should be deleted from a clean public repository. The GitHub-ready package accompanying this translated edition removes it.
3. `netstack/helper_main_stub.go` is an intentional non-Android build-tag stub used so host-side Go tooling can resolve symbols outside Android cross-compilation.
4. Historical vault derivators coexist with `PersistentVaultKeys` only for one-way migration compatibility; active writes use the current generation.
5. Optional/experimental cgo socket-binding sources are not the normal production packet transport and do not change the no-JNI hot-path architecture.

End of translated edition

This translated edition is intended to make the German master architecture dossier accessible to an English-speaking audience. For security decisions and current implementation facts, always verify against the repository source, `SECURITY.md`, `THREAT-MODEL.md`, `TOOLCHAINS.lock`, release gates and the exact artifact being deployed.